ActiMM: Worklist-Driven Sparse Convolution Runtime on GPUs

Anonymous Author(s)

Abstract

Sparse convolution is the dominant primitive for 3D point-cloud perception, yet existing GPU runtimes face a fundamental tradeoff between *computation skipping* and *execution efficiency*. Gather-GEMM-scatter approaches skip empty neighbors but suffer from per-offset launch overhead and load imbalance. Implicit GEMM approaches fuse all offsets into one efficient kernel but cannot skip empty neighbors, wasting compute proportional to sparsity.

We present ActiMM, a sparse convolution runtime built on *worklist-driven execution*. ActiMM precomputes all active neighbor pairs, groups them by kernel offset, and flattens the result into a contiguous tile stream executed by a single persistent-thread kernel. Empty offsets are skipped entirely; the stride loop provides automatic global load balance; and each tile is a pure dense GEMM. To recover data locality without heavy scheduling, ActiMM applies offset-major contiguous tile assignment and autotuned weight-stationary micro-kernels.

We evaluate ActiMM end-to-end on 3D object detection and semantic segmentation models across real-world point-cloud datasets. ActiMM achieves **TODO**× end-to-end speedup over TorchSparse++ on **TODO** and **TODO**× over SpConv v2 on **TODO**, while producing bit-identical results under FP32.

1 Introduction

Sparse convolution has become a key computational primitive in 3D point cloud perception, playing a central role in deep learning applications such as autonomous driving, robotics, and AR/VR. The importance of sparse convolution stems from the intrinsic data spatial sparsity of these applications: input data features are represented on high-dimensional voxel grids, while only a tiny fraction of voxels are active. For example, in LiDAR-based detection benchmarks such as KITTI, the active ratio is often around 0.01%, and even in indoor scenes or CAD models, it is typically only about 1%. This sparsity implies that directly applying dense convolution would waste substantial computation and memory on empty regions. Therefore, the central challenge in sparse convolution is not the convolution operation itself, but how to efficiently identify, organize, and execute the small number of irregularly distributed, input-dependent computations on GPUs. Efficient sparse convolution is thus essential for satisfying the stringent latency requirements of real-world deployment.

Unlike sparse matrix multiplication, sparse convolution is challenged not only by sparsity itself but by the fact that its computation is organized around runtime-defined spatial neighborhoods. For example, a 3×3×3 convolution has its each output location associated with 27 possible spatial offsets, but under low-density inputs, most offsets at most locations correspond to empty neighbors. Computation therefore occurs only on realized input–output adjacencies, or pairs, where an active output voxel finds an active input voxel at a given offset. Each offset shares a dense weight matrix across all such pairs, so the operator can be viewed as repeatedly applying offset-specific shared weights over a dynamically generated set of spatial pairs. Crucially, these adjacencies are not statically known but are induced by the runtime spatial distribution of active voxels. As a result, sparse convolution must simultaneously handle sparse computation, dynamic neighborhoods, indirect memory accesses, and highly uneven workload distribution, which together make efficient GPU execution fundamentally difficult.

Existing GPU sparse convolution engines can be broadly categorized into two execution strategies, including output-stationary and weight-stationary. Weight-stationary engines such as MinkowskiEngine partition computation by kernel offset, dispatching a dedicated matrix multiplication over only the valid input–output pairs at each offset. This avoids wasted arithmetic on empty voxels and the corresponding weight slice is naturally reused across all pairs within each offset. However, the number of active pairs varies widely across offsets due to intrinsic spatial sparsity, fragmenting the workload into many unevenly sized tasks that cannot saturate the GPU uniformly and leaving computational throughput well below the hardware peak. In contrast, output-stationary engines such as SpConv v2 and TorchSparse++ mirror the implicit-GEMM of dense convolution. Each thread block iterates over all kernel offsets to accumulate partial sums for its assigned output voxels in place, delivering higher throughput with more uniform occupancy. This regularity, however, means every potential neighbor position is processed uniformly, including the many that are empty, so a large fraction of multiply accumulate operations produce and discard zeros.

Overall, although the two strategies suffer from ostensibly different bottlenecks, neither succeeds in converting the GPU's raw execution capability into effective throughput, i.e., the fraction of GPU peak throughput that contributes to valid input-output pairs. Weight-stationary engines perform almost exclusively effective arithmetic, yet their raw throughput is held back by an inability to uniformly distribute work across the GPU, leaving effective throughput low in absolute terms (xx to xx% of the peak throughput

in our evaluation). Output-stationary engines sustain high raw throughput, but waste a significant share of it on empty neighbor positions, so their effective throughput remains limited (xx to xx% of the peak throughput in our evaluation). The core challenge of low-density sparse convolution is therefore not to maximize raw throughput alone, but to elevate effective throughput.

To improve effective GPU throughput, a natural alternative is to discard the structural assumptions of conventional convolution and instead extract all active input–output pairs into a single and uniform execution stream. Because only nonzero pairs are scheduled, this strategy avoids both the workload fragmentation of the weight-stationary strategy and the wasted computation of the output-stationary strategy. The collected pairs are then partitioned into tiles, i.e., equally-sized chunks of work each assigned to one GPU thread block, enabling even distribution across thread blocks for global load balance. However, interleaving tiles with different kernel offsets destroys weight reuse between adjacent tiles and prevents partial sums for the same output voxel from being accumulated in registers, thereby requiring multiple scattered atomic updates to global memory. Fundamentally, this stems from the GPU execution model: each thread block processes a single tile and retires, while the hardware scheduler alone determines where subsequent tiles execute. The result is that, although the total arithmetic is by construction proportional to the number of active pairs, the realized effective throughput is bounded not by compute capacity but by locality losses on the weight, input, and output sides. Counterintuitively, this bottleneck intensifies as the active ratio increases: a higher active ratio implies more simultaneously live kernel offsets and output locations, thereby amplifying the cost of each missed reuse opportunity.

Based on the above insights, we propose ActiMM, a worklist-driven sparse convolution runtime that computes on genuinely active input–output pairs while navigating the trade-off between global load balance and data locality recovery. ActiMM begins by explicitly enumerating every valid pair across all kernel offsets and flattening them into a single, unified worklist. The worklist is then partitioned into fixed-size tiles, each of which is a batch of pairs from one offset and serves as the basic GPU scheduling unit. A persistent-thread execution model in ActiMM lets each thread block consume multiple tiles in stride, enabling locality-aware reordering without disrupting the tiled execution strategy. ActiMM further employs an offline analyzer to select the optimal locality optimization strategy for different input activation ratios. At low active ratios, offset-major ordering arranges tiles by kernel offset so that consecutive tiles processed by the same thread block can reuse weights, restoring weight locality with negligible overhead. At high active ratios, static bundling groups tiles from different offsets that contribute to the same output positions, promoting an output-stationary execution pattern and reducing redundant global memory traffic.

Experimental results show ...

Our contribution

References